CLAMP: Fairness-Constrained Federated Graph Learning from Validated Sums with Private Group Statistics

Ngoc-Son-An Nguyen[a], Quang-Vinh Dang[b,*]

[a]*Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam*
[b]*British University Vietnam, Hung Yen, Vietnam*

Abstract

Federated graph neural networks are trained by institutions that cannot pool their graphs, and the resulting models should be fair across demographic groups, should not expose sensitive attributes, and should withstand a minority of compromised clients. Existing methods provide these properties with mechanisms that inspect individual clients, which the server must be trusted with and an adversary can target. We present Clamp, a protocol in which the server only ever sees validated sums. Clients clip their model updates and report four group statistics computed without reading the sensitive attribute; secure aggregation with norm and range checks sums them, Gaussian noise privatises the statistics, and a single Lagrange multiplier broadcast to all clients enforces a global demographic-parity budget. The server returns the latest model, after a warm-up, whose released gap met the budget. We prove attribute-level differential privacy for validation nodes in the statistics channel, a per-round bound on the influence of Byzantine updates, a bound on the influence of falsified statistics on the fairness estimate, and a fairness guarantee for the returned model on honest validation data, and we state the scope of each; the two Byzantine bounds are loose, and in many settings vacuous. On eight benchmarks with ten seeds against eleven baselines, Clamp brings the demographic-parity gap from 0.071, 0.042 and 0.052 to 0.018, 0.026 and 0.028 on the three graphs where the budget of 0.03 binds,

*Corresponding author. ORCID: 0000-0002-3877-8024

Email addresses: annns25871@pgr.iuh.edu.vn (Ngoc-Son-An Nguyen), vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

at an AUC within 0.01 of an unconstrained clipped model, and the budget acts as a control knob within a dataset-dependent range. Baselines that read per-client fairness information in the clear reach lower gaps. On small graphs the private statistics are too noisy for effective control, and a single client that falsifies its statistics can switch the controller off.

Keywords: Federated learning, Graph neural networks, Group fairness, Differential privacy, Byzantine robustness, Secure aggregation

1. Introduction

Graph neural networks (GNNs) [1, 2] are used for credit scoring, bail decisions and social recommendation, all settings in which the data are split across institutions and the predictions affect people. Federated learning [3] lets such institutions train one model without pooling their graphs. Three requirements then arise together. The model should not discriminate between demographic groups [4, 5]. The federation should not learn more about any client, or about any person's sensitive attribute, than it needs to [6, 7]. And a few compromised clients should not be able to steer the model [8, 9, 10].

These requirements are usually addressed one or two at a time, and the mechanisms do not compose. Fair federated methods reweight clients or calibrate the model using per-client fairness or loss reports [11, 12, 13]; the server must read those reports, and a Byzantine client can forge them to capture weight or to hide unfairness. Byzantine-robust aggregation rules such as the coordinate median, the trimmed mean and Krum [9, 8] need to see individual updates, so they cannot run under secure aggregation [14], the standard tool for hiding individual updates from the server. Private fair methods [15, 16] add noise to gradients but assume every client is honest. In our earlier work on federated GNNs for fraud detection [17] we combined privacy and fairness through client-side mechanisms, and the same tension appears there: every component that inspects one client at a time is a component the server must be trusted with and an adversary can target.

This paper takes a different route. We ask what can be achieved if the server is only ever allowed to see *sums over clients*, each of whose summands is *range-checked*. We show that this restriction is enough to enforce a global group-fairness constraint and to privatise the information the constraint needs, and we quantify how far it bounds the influence of Byzantine clients: per round on the model, and, much more weakly, on the fairness

estimate. The resulting protocol, CLAMP, has three components:

- **Clipped secure aggregation of updates.** Clients clip their updates to norm C, and secure aggregation with a validity check rejects any update above the bound. The server sees only the sum.
- **Private group statistics.** With each release, clients report four scalars: per group, the sum of predictions and the count of validation nodes. These are computed with a forward pass that never reads the sensitive attribute, so their sensitivity is a constant, and they are summed under secure aggregation with Gaussian noise.
- **One global Lagrange multiplier.** From the noisy sums, the server estimates the global demographic-parity gap and updates a single multiplier by projected dual ascent. It broadcasts the multiplier and the sign of the gap. Clients add a signed penalty to their local loss.

The design has no per-client weights, scores or reputations that an adversary could capture over time, and every server-side computation is compatible with secure aggregation. It does not, however, protect the fairness statistics against a minority that falsifies them, as our experiments show.

Contributions.

1. We formulate fairness-constrained federated graph learning under a server that sees only validated sums and an adversary that controls up to f clients' updates *and* fairness statistics (Section 3).
2. We propose CLAMP, a protocol that enforces a global demographic-parity budget through a single broadcast multiplier computed from privatised aggregate statistics (Section 4).
3. We prove five results with explicit scope (Section 5): attribute-level differential privacy for validation nodes in the statistics channel, a per-round bound $2fC/(K-f)$ on Byzantine influence on the model, a bound on the influence of falsified statistics on the fairness estimate, a fairness guarantee for the returned model on honest validation data, and an a-posteriori record of the average estimated constraint violation. We state what each result does not cover; in particular, the two Byzantine bounds are loose, and in many of our settings vacuous.

4. We evaluate CLAMP on eight fairness benchmarks against eleven baselines with ten seeds and paired significance tests, and under update attacks, statistics-falsification attacks, a range of privacy budgets, and ablations (Sections 6–7). The results show where the constraint is met, what it costs in utility, and where the approach is weak: on small graphs the noisy statistics limit control, and the clipping that bounds Byzantine influence also costs some accuracy.

2. Related work

Fair graph neural networks. Centralised fair GNNs remove sensitive information from representations or rebalance training: FairGNN [4] uses an adversary to estimate and remove the attribute, NIFTY [5] enforces counterfactual stability, EDITS [18] debiases attributes and structure, and FairSIN [19] neutralises sensitive information through added features; re-balancing [20] and attribute-agnostic methods [21] follow the same goal. All of them assume one party holds the whole graph and the attribute. We use FairGNN and FairSIN as baselines, trained with federated averaging.

Fairness in federated learning. FairFed [11] reweights clients by the gap between their local fairness and the global fairness, q-FFL [12] reweights by local loss, and FedFACT [13] calibrates a trained model to a group-fairness target with provable control. PUFFLE [15] and FedFDP [16] combine fairness with differential privacy. FedFDP targets *performance* fairness across clients rather than group fairness, which we note when we compare against it. For graphs, F^2GNN [22] combines structure-aware local fairness with fairness-weighted aggregation, FairGFL [23] handles overlapping subgraphs under privacy, and FedGraph-Fair [24] uses personalisation and distributionally robust optimisation. These methods either read per-client fairness or loss values on the server or assume honest clients. CLAMP uses only validated sums and is designed for a Byzantine minority.

Constrained learning. Enforcing fairness as a constraint through a Lagrangian is standard in centralised learning [25]. Our dual update is the simplest form of this approach, with one multiplier. What is new is where the information comes from: the multiplier is driven by statistics that are privatised, aggregated under secure aggregation and partly controlled by an adversary, and we bound the effect of each of these on the estimate.

Byzantine robustness and secure aggregation. Robust aggregation rules (Krum [8], coordinate median and trimmed mean [9], Bulyan [26]) and trust-scored rules such as FLTrust [27] require access to individual updates, and they can be evaded by attacks that stay within the spread of honest updates [10, 28]. Secure aggregation [14] hides individual updates, which rules these defences out. Verifiable aggregation systems such as Prio [29] and RoFL [30] restore a limited form of robustness by proving norm and range constraints on encrypted inputs. CLAMP builds on exactly this capability: its only robustness mechanism is a norm and range check, and we show what that check guarantees for both the model and the fairness estimate. Distributed differential privacy with secure aggregation [31] provides the noise mechanism we assume in deployment.

Positioning. Previous methods for federated graph learning are fair, private or robust, and the ones that combine two properties mostly do so with mechanisms that inspect individual clients. A closely related line combines fairness-aware federated aggregation with cryptographic protection of the reports: Bendoukha et al. [32] compute FairFed's weights under homomorphic encryption, which hides the reports but still uses per-client fairness values and assumes honest clients. Byzantine-resilient secure aggregation protocols such as BREA [33] and ELSA [34] provide stronger update-level robustness under encryption than a norm check, at a higher cryptographic cost, but do not address fairness. CLAMP is a simpler point in this space: it obtains fairness control, a privacy guarantee for the statistics and a bound on per-round update influence from validated sums alone, and our experiments show where these properties are weak. Our own earlier conference work [17] addressed privacy and fairness for fraud detection with client-side mechanisms and did not consider Byzantine clients or secure aggregation.

3. Problem setting and threat model

Federated node classification. K clients each hold a subgraph $G_k = (V_k, E_k)$ with node features X_k, binary labels $y_v \in \{0, 1\}$ and a binary sensitive attribute $s_v \in \{0, 1\}$. Every client splits its labelled nodes into training, validation and test sets. A graph neural network f_θ maps each node to a predicted probability $p_\theta(v) \in (0, 1)$. The sensitive attribute is *not* a feature: it is removed from X_k before training and the backbone never reads it. The server coordinates T rounds of training of a single global model θ.

Fairness target. We use the soft demographic-parity gap of the global model on the union of the clients' nodes,

$$D(\theta) = \Big| r_1(\theta) - r_0(\theta) \Big|, \qquad r_a(\theta) = \frac{\sum_k \sum_{v \in V_k,\, s_v = a} p_\theta(v)}{\sum_k |\{v \in V_k : s_v = a\}|}, \tag{1}$$

and the goal is to train a model whose test-set gap stays below a budget τ chosen by the federation, at the smallest possible cost in utility:

$$\min_\theta \; F(\theta) = \sum_k \tfrac{n_k}{n} F_k(\theta) \quad \text{s.t.} \quad D(\theta) \le \tau. \tag{2}$$

The constraint is *global*. Under non-IID partitions a client's local gap can differ in sign and magnitude from the global one, so neither local fairness regularisation nor averaging per-client gaps solves (2).

What the server may see. The server is honest-but-curious about individual clients. It must not learn an individual client's update or an individual client's group statistics, and the released aggregate statistics must not reveal any individual node's sensitive attribute beyond a differential-privacy budget.

Byzantine clients. Up to $f < K/2$ clients may be Byzantine. They know the protocol, the public constants and the current global state, and may coordinate. They may submit arbitrary model updates and arbitrary group statistics, with one restriction: a submission must pass the validity check of the secure-aggregation layer, which bounds update norms and requires the statistics to describe a possible validation set (Section 4). Submissions that fail the check are discarded. We consider two goals of the adversary: to degrade or steer the model (*update attacks*), and to make the federation believe the model is fairer than it is (*statistics attacks*), so that the fairness constraint is relaxed.

Why existing designs do not meet this setting. Methods that weight clients by locally computed fairness or loss values [11, 12] need per-client reports, which the server can read and a Byzantine client can forge at will. Byzantine-robust rules such as the coordinate-wise median or Krum [9, 8] need the individual updates and are therefore incompatible with secure aggregation. Private fair FL methods [15, 16] protect the gradients but assume honest clients. CLAMP is designed so that the server only ever needs *sums*.

4. The Clamp protocol

Clamp has three ideas. First, every quantity the server uses is a *sum over clients* of a *range-checked* summand, so it can be computed inside secure aggregation and no single client can move it by more than a known amount. Second, the fairness constraint is enforced by *one* global Lagrange multiplier λ and *one* global direction d, both computed from noisy aggregate group statistics and broadcast to all clients; there is no per-client weight that could be captured. Third, the statistics are computed with a forward pass that never reads the sensitive attribute, which gives them a small, data-independent sensitivity and makes them cheap to privatise. Algorithm 1 lists one round.

4.1. Client side

At round t the server broadcasts $(\theta^t, \lambda^t, d^t)$ with $\lambda^0 = 0$ and $d^0 = 0$.

Group statistics. Client k evaluates the received model on its validation nodes and reports, for $a \in \{0, 1\}$,

$$N_{k,a} = \Big|\{v \in \mathcal{V}_k^{\text{val}} : s_v = a\}\Big|, \qquad S_{k,a} = \sum_{v \in \mathcal{V}_k^{\text{val}},\, s_v = a} p_{\theta^t}(v). \tag{3}$$

Because s is not an input of the backbone, flipping one node's attribute changes no prediction; it only moves that node's prediction and count from one group to the other. A public constant n_{cap} bounds $N_{k,a}$; it is fixed per dataset before training (Section 6). A client whose group is larger than n_{cap} reports a fixed subset chosen once, so every report satisfies $0 \le S_{k,a} \le N_{k,a} \le n_{\text{cap}}$.

Lagrangian local objective. Starting from θ^t, the client runs E local epochs on its training nodes with

$$\mathcal{L}_k(\theta) = \text{BCE}_w\Big(p_\theta, y\Big) + \lambda^t d^t \Big(m_{k,1}(\theta) - m_{k,0}(\theta)\Big), \tag{4}$$

where BCE_w is class-weighted binary cross-entropy and $m_{k,a}$ is the mean prediction over the client's training nodes in group a (zero for an empty group). The penalty is *signed*: d^t is the sign of the estimated *global* gap, so each client pushes the two group rates toward each other in the direction the federation needs, even when its own local gap has the opposite sign. With $\lambda^t = 0$ or $d^t = 0$, (4) is ordinary weighted cross-entropy.

Clipping. The client forms the pseudo-gradient $\Delta_k = \theta^t - \theta_k$ and clips it, $\Delta_k \leftarrow \Delta_k \min\bigl(1, C/\|\Delta_k\|_2\bigr)$. It submits $(\Delta_k, S_{k,0}, S_{k,1}, N_{k,0}, N_{k,1})$ to secure aggregation.

4.2. Verifiable secure aggregation

A submission is *valid* if

$$\|\Delta_k\|_2 \le C \quad \text{and} \quad 0 \le S_{k,a} \le N_{k,a} \le n_{\text{cap}} \quad (a = 0, 1). \tag{5}$$

Invalid submissions are dropped from every sum, and the number of accepted submissions K' is public. In deployment, the sums are computed with secure aggregation [14] and validity is enforced with zero-knowledge norm and range proofs, as in Prio [29] and RoFL [30], so the server learns the sums and K' but no individual summand. Our experiments simulate this layer in plaintext: the server code applies (5) and then reads only the sums. The cost of the cryptographic layer is not part of our evaluation.

4.3. Server side

If $K' = 0$ the round is skipped. Otherwise the server receives

$$U = \sum_{k \in A} \Delta_k + \xi_u, \qquad S_a = \sum_{k \in A} S_{k,a} + \xi_{S,a}, \qquad N_a = \sum_{k \in A} N_{k,a} + \xi_{N,a}, \tag{6}$$

where A is the set of accepted clients, $\xi_{S,a}, \xi_{N,a} \sim \mathcal{N}(0, \sigma_s^2)$ are independent, and $\xi_u \sim \mathcal{N}(0, \sigma_u^2 I)$ is optional ($\sigma_u = 0$ unless stated). It then computes

$$r_a = \text{clip}\bigl(S_a / \max(N_a, 1),\, 0,\, 1\bigr), \qquad g = r_1 - r_0, \qquad \widehat{D} = |g|, \tag{7}$$

$$\theta^{t+1} = \theta^t - U/K', \tag{8}$$

$$\lambda^{t+1} = \max\bigl(0,\ \lambda^t + \eta\,(\widehat{D} - \tau)\bigr), \tag{9}$$

$$d^{t+1} = \begin{cases} \text{sign}(g) & \text{if } |g| > \tau/2, \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

Equation (9) is projected dual ascent with a constant step η: the multiplier grows while the estimated gap exceeds the budget and decays otherwise. The dead zone in (10) stops the fairness push once the gap is well inside the budget, which avoids chattering around zero.

Output rule. The signed penalty pushes the group rates toward each other with a force that does not shrink as the gap closes, so the gap tends to oscillate around the budget and the last iterate lands at an arbitrary point of the oscillation. CLAMP therefore does not return the last model. In every release round after a warm-up of W rounds the server stores the model θ^t on which the released statistics were computed whenever its released estimate satisfies $\widehat{D}^t \leq \tau$, and at the end it returns the most recently stored model, or the last model if no eligible round was feasible. The warm-up excludes the first rounds, whose barely trained models predict almost the same probability for every node and are therefore trivially fair; we use $W = \lceil 0.2T \rceil$. The rule uses only released quantities, so it is post-processing and costs no privacy (Theorem 1); Proposition 4 states what it guarantees.

Release period. On small graphs the statistics noise can dominate the gap estimate. CLAMP therefore releases statistics, and updates (λ, d), only every P rounds; in the other rounds clients keep the last broadcast (λ, d) and the server aggregates updates only. Privacy is then accounted over $\lceil T/P \rceil$ releases (Theorem 1).

Cost. Relative to FedAvg, each client sends four extra scalars per release round and computes one extra forward pass on its validation nodes. The server's work is unchanged apart from four scalar sums. There is no per-client computation on the server, so apart from the cryptographic layer the protocol scales with K exactly as FedAvg does. The cryptographic layer is not free: zero-knowledge norm proofs over d-dimensional updates dominate the cost of verifiable secure aggregation in systems such as RoFL [30], and we do not benchmark it.

5. Guarantees

This section states what CLAMP guarantees and, just as explicitly, what it does not. Each result covers one channel of the protocol. None of them is a convergence theorem for a non-convex GNN, and Section 7 shows that several of the bounds are loose or vacuous in the settings we evaluate.

5.1. Privacy of validation-node attributes in the statistics channel

Definition 1 (Attribute-neighbouring datasets)**.** Two federated datasets are *neighbouring* if they are identical except for the sensitive attribute s_v of a single validation node v of one client.

Algorithm 1 One round t of CLAMP

Require: θ^t, λ^t, d^t; constants $C, \tau, \eta, n_{\text{cap}}, \sigma_s, \sigma_u, P$

Server broadcasts $(\theta^t, \lambda^t, d^t)$
for each client k **in parallel do**
  if $t \bmod P = 0$ **then**
    compute $(S_{k,a}, N_{k,a})_{a=0,1}$ with θ^t on validation nodes, Eq. (3)
  end if
  $\theta_k \leftarrow E$ local epochs on $\mathcal{L}_k$, Eq. (4), from θ^t
  $\Delta_k \leftarrow (\theta^t - \theta_k)\min(1, C/\|\theta^t - \theta_k\|_2)$
  submit Δ_k (and the statistics) to secure aggregation
end for
Secure aggregation drops invalid submissions, Eq. (5), and outputs the noisy sums, Eq. (6)
if $K' = 0$ **then**
  return $(\theta^t, \lambda^t, d^t)$
end if
$\theta^{t+1} \leftarrow \theta^t - U/K'$
if $t \bmod P = 0$ **then**
  compute $g, \widehat{D}$ by Eq. (7); update λ^{t+1} by Eq. (9) and d^{t+1} by Eq. (10)
  if $t + 1 \geq W$ **and** $\widehat{D} \leq \tau$ **then** store $\hat{\theta} \leftarrow \theta^t$ {output rule}
else
  $(\lambda^{t+1}, d^{t+1}) \leftarrow (\lambda^t, d^t)$
end if
after round T**:** return the stored $\hat{\theta}$ (or θ^T if none was stored)

The relation protects the quantity the statistics are about, namely group membership, for validation nodes only. Training nodes are not covered: their attributes enter the local objective (4). Test nodes are used by no part of the protocol. Throughout, n_{cap} is a public constant that does not depend on any attribute: the largest per-client validation-set size, rounded up to a multiple of 50. Since no group of a client can be larger than the client's validation set, no report is ever truncated.

Theorem 1 (Differential privacy of the released statistics)**.** *Let $\sigma_s = 2z$ and let the statistics be released in $R = \lceil T/P \rceil$ rounds. Fix all training data and all randomness of local training. Then the sequence of released noisy statistics (S_0, S_1, N_0, N_1) over all release rounds satisfies $(\alpha, 2R\alpha/\sigma_s^2)$-*

Rényi differential privacy [35] for every $\alpha > 1$ with respect to attribute-neighbouring datasets, and hence (ε, δ)-DP with $\varepsilon = \min_{\alpha>1}\Big[R\alpha/(2z^2) + \log\frac{\alpha-1}{\alpha} - \frac{\log\delta+\log\alpha}{\alpha-1}\Big]$ [36]. The same holds for every quantity the server computes, including λ^t, d^t, the models θ^t and the returned model.

Proof. Consider one release round. Neighbouring datasets differ in s_v for one validation node v. The backbone does not read s, local training reads only training-node attributes, and the cap never truncates, so every prediction and every other node's group membership are identical in the two datasets. The prediction $p_v \in [0, 1]$ moves from one group's sum to the other's and one unit moves between the two counts. The client's vector (S_0, S_1, N_0, N_1) therefore changes by $(\pm p_v, \mp p_v, \pm 1, \mp 1)$, whose ℓ_2 norm is $\sqrt{2p_v^2 + 2} \le 2$. The changed report still satisfies $0 \le S_a \le N_a \le n_{\text{cap}}$, so the validity check accepts or rejects the same set of clients, and every other client's contribution is unchanged. Each coordinate receives independent $\mathcal{N}(0, \sigma_s^2)$ noise, so one release is a Gaussian mechanism with ℓ_2 sensitivity 2, which is $(\alpha, 2\alpha/\sigma_s^2)$-RDP. The input to a release depends on the attribute of v only through (λ, d) and the models, and with the training data fixed these depend on it only through earlier releases. Adaptive composition of RDP over R releases therefore gives $(\alpha, 2R\alpha/\sigma_s^2)$-RDP, and everything the server computes from the releases is post-processing. No subsampling amplification is used, since the same validation nodes are used in every release. □

Remark 1 (Scope). Theorem 1 protects the attribute of validation nodes, a quarter of the labelled nodes in our splits. It does *not* make the update channel differentially private, since the update sum U depends on training-node attributes through the fairness penalty. That channel is hidden from the server by secure aggregation, which is a cryptographic guarantee about individual updates, not a bound on what the aggregate reveals. At the default $\varepsilon = 8$ the guarantee is also weak as a bound on an attacker's success (Section 7.5). Setting $\sigma_u > 0$ adds Gaussian noise to U; with sensitivity C under adding or removing one client this is a Gaussian mechanism. The values of σ_u in our ablation are far too small to give a meaningful privacy level, and we make no client-level claim.

Remark 2 (Where the noise is added). In our simulation the noise is added once to the real-valued sum, which corresponds to a trusted aggregator. A secure-aggregation deployment works over a finite group. Each client then quantises its statistics and adds discrete Gaussian noise [36, 31]. Quantisation

changes the sensitivity, and the sum of discrete Gaussians is not exactly a discrete Gaussian, so the accounting of Kairouz et al. [31] replaces the one above. Each client adds a variance share of $\sigma_s^2/(K-f)$ so that the sum carries at least σ_s^2 while at most f clients withhold their share.

5.2. Per-round influence of Byzantine updates

Theorem 2 (Per-round influence on the model)**.** *Fix the state $(\theta^t, \lambda^t, d^t)$ and let $\sigma_u = 0$. Let A be the accepted set, $K' = |A|$, $B \subseteq A$ the accepted Byzantine clients with $|B| = f' \le f$, and $H = A \setminus B$. If every honest client is accepted, then*

$$\left\| \tfrac{U}{K'} - \tfrac{1}{|H|} \textstyle\sum_{k \in H} \Delta_k \right\|_2 \;\le\; \tfrac{2f'C}{K'} \;\le\; \tfrac{2fC}{K-f}. \tag{11}$$

Proof. Write $\bar{\Delta}_H$ and $\bar{\Delta}_B$ for the means over H and B. Then $U/K' - \bar{\Delta}_H = \frac{f'}{K'}(\bar{\Delta}_B - \bar{\Delta}_H)$. Every accepted update has norm at most C, so $\|\bar{\Delta}_B - \bar{\Delta}_H\|_2 \le 2C$. Finally $K' \ge |H| \ge K - f$ and $f' \le f$. □

The bound only says that each Byzantine client moves the average by at most a fixed amount per round. It is the same bound that any clipped mean satisfies, and it is not small. Once $f \ge K/3$ it reaches the norm C of a full honest step, for example $1.33C$ at $f = 4$ of $K = 10$. It is per round and against a same-state counterfactual. It does not compose over rounds, because later honest updates are computed at the perturbed model and the adversary can also act through the statistics, and we make no multi-round claim. A rule that inspects individual updates (median, Krum) can do better against some attacks, but it cannot run on sums. Section 7.3 measures what the bound means in practice.

5.3. Influence of falsified statistics on the fairness estimate

Let S_a^H, N_a^H be the honest totals and $D^H = |S_1^H/N_1^H - S_0^H/N_0^H|$ the honest gap.

Theorem 3 (Influence of falsified statistics)**.** *Let $\sigma_s = 0$ and assume every honest client is accepted and $N_a^H > 0$. For any statistics submitted by at most f Byzantine clients that pass the validity check,*

$$\left|\widehat{D} - D^H\right| \le b_0 + b_1, \qquad b_a = \frac{f\, n_{\mathrm{cap}}}{N_a^H + f\, n_{\mathrm{cap}}}. \tag{12}$$

With noise, let $r = \sigma_s\sqrt{2\ln(8/\beta)}$ *and assume* $N_a^H > 3r$*. Then with probability at least* $1-\beta$ *over the noise of one release,*

$$\left|\widehat{D} - D^H\right| \le \sum_{a\in\{0,1\}} \left[b_a + \frac{2r}{N_a^H - r}\right]. \tag{13}$$

On this event, the estimated direction d^{t+1} *equals the honest direction whenever* $|g^H|$ *exceeds* $\tau/2$ *plus the right-hand side.*

Proof. Let $S_a^B \in [0, N_a^B]$ and $N_a^B \in [0, fn_{\text{cap}}]$ be the accepted Byzantine totals. The aggregate rate is a convex combination, $\frac{S_a}{N_a} - \frac{S_a^H}{N_a^H} = \frac{N_a^B}{N_a^H+N_a^B}\left(\frac{S_a^B}{N_a^B} - \frac{S_a^H}{N_a^H}\right)$. The difference of two rates in $[0,1]$ is at most 1 in absolute value, and $x \mapsto x/(N_a^H + x)$ is increasing, so the rate error is at most b_a. The absolute value and the clip to $[0,1]$ are 1-Lipschitz, which gives (12). With noise, a Gaussian tail bound and a union bound over the four noise terms give $|\xi| \le r$ for all of them with probability at least $1-\beta$. Since $|\frac{S+\xi_S}{N+\xi_N} - \frac{S}{N}| \le \frac{|\xi_S|+|\xi_N|}{N-|\xi_N|}$ for $0 \le S \le N$ and $|\xi_N| < N$, on that event each rate is additionally perturbed by at most $2r/(N_a^H - r)$; here the condition $N_a^H > 3r$ keeps the denominators positive and the term below 1. The direction claim follows because g and g^H then have the same sign and $|g| > \tau/2$. □

The bound is tight for a single round, since an adversary that reports $N_a^B = fn_{\text{cap}}$ with an extreme rate attains it. But it is large whenever fn_{cap} is comparable to the honest validation mass of a group, and with n_{cap} set by the largest client's validation set this is the case in most of our settings. On Credit, whose protected group is 9% of the nodes, the bound exceeds 1 for every $f \ge 1$, and since both $\widehat{D}$ and D^H lie in $[0,1]$ it is then vacuous (Table 3). The bound depends on the honest mass N_a^H, which the server cannot observe. An observable but looser version replaces N_a^H by $\max(N_a - fn_{\text{cap}} - r, 1)$. Theorem 3 should therefore be read as a statement of how much a falsifying minority can move the estimate, not as a guarantee that it cannot move it much. The experiments confirm that a single falsifying client can switch the controller off. The dead zone in (10) does not help here: an adversary only needs to push $|g|$ into the dead zone to stop the fairness push. A bounded-influence design for this channel would need per-client caps tied to attested validation-set sizes, which we leave open. Because there are no per-client weights, a Byzantine client also cannot gain influence over time by building reputation.

Proposition 4 (Fairness of the returned model)**.** *Let $\hat{\theta}$ be the model returned by the output rule, selected at release round $t^\star$, and let $D^H(\hat{\theta})$ be the gap of $\hat{\theta}$ on the honest clients' validation nodes. Assume the conditions of Theorem 3 hold at every release round, with $r_R = \sigma_s\sqrt{2\ln(8R/\beta)}$ in place of r and $N_a^H > 3r_R$. Then, with probability at least $1-\beta$ over the noise of all R releases, either the rule falls back to the last model or*

$$D^H(\hat{\theta}) \le \tau + \sum_{a\in\{0,1\}} \Big[b_a + \frac{2r_R}{N_a^H - r_R} \Big]. \tag{14}$$

Proof. The round $t^\star$ is chosen using the noisy estimates, so the single-round bound of Theorem 3 cannot be applied at $t^\star$ directly. Apply it with failure probability β/R at each of the R release rounds. By a union bound, with probability at least $1-\beta$ the bracketed deviation bound holds at every release round simultaneously, and in particular at $t^\star$. By construction $\widehat{D}^{t^\star} \le \tau$, and $\widehat{D}^{t^\star}$ was computed on $\hat{\theta}$. □

The guarantee concerns the validation gap of the honest clients, not the test gap. The difference between the two is a generalisation gap that no protocol-level argument controls, and we measure it in Section 7. It inherits the looseness of Theorem 3 when Byzantine clients are present. With $f = 0$ only the noise term remains, and the proposition then says that the returned model met the budget on honest validation data up to the noise. We report how often the rule falls back to the last model.

5.4. An a-posteriori record of the dual update

Proposition 5 (Average estimated violation)**.** *For any sequence of models and estimates, the dual update* (9) *with $\lambda^{(0)} = 0$ satisfies, over the release rounds $j = 0, \dots, R-1$,*

$$\frac{1}{R}\sum_{j=0}^{R-1} \big(\widehat{D}^{(j)} - \tau\big) \le \frac{\lambda^{(R)}}{\eta R}. \tag{15}$$

Proof. $\lambda^{(j+1)} = \max(0, \lambda^{(j)} + \eta(\widehat{D}^{(j)} - \tau)) \ge \lambda^{(j)} + \eta(\widehat{D}^{(j)} - \tau)$. Sum over $j = 0, \dots, R-1$ and use $\lambda^{(0)} = 0$. □

Proposition 5 is an elementary consequence of the projection, not a convergence result. It bounds the average of the noisy estimates over all release

rounds. It says nothing about the returned model, and it bounds the estimated validation gap, which is noisy and biased upward by the absolute value, not the test gap. Its right-hand side is logged, and we report it for every CLAMP configuration in the appendix (Table A.9). On Credit its mean value of 0.008 certifies an average estimated gap of at most 0.038, slightly above the budget. Under the textbook assumptions of a convex loss and constraint, exact per-round minimisation and noiseless statistics, (9) is projected supergradient ascent on a concave dual with the usual $O(1/\sqrt{R})$ rate [25]. None of these assumptions holds for a GNN, and we do not rely on them.

6. Experimental setup

Datasets. We use eight public benchmarks for fair node classification (Table 1), all with a genuine binary sensitive attribute. *German* (credit risk; gender) and *Credit* (credit default; age) are similarity graphs over individuals, and *Bail* (bail decisions; race) links defendants with similar records [5]. *Pokec-z* and *Pokec-n* (region) are the two regional subsets of the Pokec social network used in [4], and *NBA* (nationality) links basketball players [4]. *Facebook* (gender) is ego network 107 of the Facebook social circles data [37], and *AMiner-S* is a co-authorship network whose sensitive attribute is the continent of the author's affiliation; we take both from the PyGDebias collection [38]. For AMiner-S the attribute has seven values, which we binarise as the most frequent continent against the rest, and we reduce the 5,693 bag-of-words features to 256 dimensions with truncated SVD (fixed random state) to keep full-batch training tractable; the label is the most frequent research field against the rest, as in PyGDebias. The sensitive attribute is removed from the node features in every dataset. Each run draws a stratified 50/25/25 train/validation/test split of the labelled nodes and partitions the nodes over $K = 5$ clients with a Dirichlet($\alpha = 0.3$) split over labels, which produces strongly non-IID clients. Each client keeps the subgraph induced by its nodes, so cross-client edges are lost (Table 1). The split, the partition and the model initialisation all depend on the seed.

Protocol constants. All methods train for $T = 100$ rounds with $E = 3$ full-batch local epochs, AdamW with learning rate 0.01 and weight decay 10^{-5}, two-layer backbones with 64 hidden units (GAT: 4 heads) and dropout 0.3. For CLAMP we use $C = 1$, $\eta = 0.5$, fairness budget $\tau = 0.03$, statistics privacy $\varepsilon = 8$ at $\delta = 10^{-5}$ for Theorem 1, and release period $P = 1$. On the

Table 1: Datasets after preprocessing. Edges are undirected. "Kept" is the fraction of edges inside a client after the $K = 5$ Dirichlet partition, and n_{cap} the public cap on a client's per-group validation count (both for seed 0). $P(y{=}1)$ is over labelled nodes.

Dataset	Nodes	Edges	Features	$P(y{=}1)$	$P(s{=}1)$	Kept	n_{cap}
German	1,000	21,742	26	0.700	0.310	0.21	50
Bail	18,876	311,870	16	0.376	0.506	0.38	1,150
Credit	30,000	1,421,858	12	0.779	0.090	0.34	3,450
Pokec-z	67,796	617,958	276	0.081	0.352	0.29	4,250
Pokec-n	66,569	517,047	265	0.064	0.289	0.29	4,300
AMiner-S	39,424	52,460	256	0.197	0.403	0.45	3,350
NBA	403	10,621	95	0.508	0.266	0.33	50
Facebook	1,045	26,749	573	0.787	0.342	0.31	100

three small graphs (German, NBA, Facebook) we use $P = 10$; this choice was made before the experiments, on the expectation that per-round noise would dominate there, and the privacy sweep in the supplementary material shows that it is not better than $P = 1$ on German for most budgets. These constants were fixed before the main experiments and were not tuned per dataset. The output rule uses a warm-up of $W = \lceil 0.2T \rceil = 20$ rounds. We added the warm-up after observing, in a first analysis of the results, that without it the rule sometimes returned the barely trained round-1 model. Because the per-round test metrics are logged, the rule with warm-up was applied exactly to all runs; it changed 49 of the runs, which are listed in the released results. n_{cap} must be public (Section 5). In our experiments it was computed per group: the largest per-client validation group size, rounded up to a multiple of 50 (Table 1). This is smaller than the public, attribute-independent cap of Section 5, the largest per-client validation-set size. It does not affect honest runs, because no report is truncated under either cap, but it makes the statistics attacks of Section 7.4 weaker than an adversary under the public cap could make them. CLAMP uses a GAT backbone, and CLAMP-GCN swaps in GCN.

Baselines. We compare with eleven baselines.

- FedAvg [3] with GCN and GAT backbones.
- The fair GNNs FairGNN [4] and FairSIN [19], with their own backbones, trained with FedAvg.

- The fair FL methods FairFed [11] and FedFACT [13] (its closed-form group mean-matching post-processing).
- The federated graph methods F^2GNN [22] (GAT), FairGFL [23] (its aggregation rule) and FedGraph-Fair [24] (its distributionally robust reweighting, without personalisation).
- The private fair methods PUFFLE [15] and FedFDP [16].

FairFed, FairGFL, FedGraph-Fair, FedFACT, PUFFLE and FedFDP use a GCN backbone, as in their original federated settings. The appendix table lists the backbone of every method. FairFed, F^2GNN and FedFACT read per-client fairness quantities or group rates in the clear on the server; CLAMP reads only noisy sums. PUFFLE and FedFDP use DP-SGD with privacy accounted per local step at $\varepsilon = 8$ for the gradients, so their privacy unit (a client's training data) and ours (a validation node's attribute) differ, and the two values of ε are not comparable. For FedFDP we clip at the batch level rather than per sample, because per-node gradients are not independent in a message-passing GNN. FedFDP optimises fairness of performance across clients, not group fairness, so it is not expected to reduce the demographic-parity gap. All baselines use the default hyperparameters of our implementations, which are documented in the released code; we did not tune them per dataset, just as we did not tune CLAMP. Two ablations isolate the parts of CLAMP. ClipMean is CLAMP with $\eta = 0$ and the last iterate, i.e. clipped secure aggregation without fairness control. CLAMP-Select is CLAMP with $\eta = 0$ and the output rule, i.e. selection without a controller.

Metrics and statistics. We report test AUC, the soft demographic-parity gap DPD of Eq. (1) on test nodes, and, in the appendix, the hard demographic-parity gap and the equal-opportunity gap EOD at threshold 0.5, together with the certificate of Proposition 5 for every CLAMP variant. The main comparison uses ten seeds per configuration. The sweeps use five seeds, and the attack studies and ablations use three. We test each baseline against CLAMP with a two-sided Wilcoxon signed-rank test paired by seed, with Holm correction across baselines within each dataset and metric. With ten seeds the smallest attainable Holm-adjusted p-value is about 0.03.

Attacks. Update attacks (Section 7.3) use $K = 10$ clients of which $f \in \{1, 2, 3, 4\}$ are Byzantine: sign flipping, scaling, ALIE [10], inner-product

manipulation [28] and a fairness-poisoning attack whose local loss rewards a larger demographic-parity gap while still fitting the labels. Under CLAMP attacker updates are clipped to the norm bound C like all others, so they always pass the validity check. We compare CLAMP with FedAvg, the coordinate median, the trimmed mean and Krum, all with a GAT backbone; the last three need individual updates and cannot run under secure aggregation. Statistics attacks (Section 7.4) let the Byzantine clients train honestly but falsify their statistics with $N_a = n_{\text{cap}}$ and either swapped group rates (*swap*) or both rates equal to the honest pooled rate (*mid*), which drives the reported gap toward zero.

Compute. German, Bail, NBA and Facebook, and the privacy studies, were run on a 4-core CPU. Credit, Pokec-z, Pokec-n and AMiner-S, and the sweep, attack and ablation studies on these graphs, were run on an NVIDIA T4 GPU. One run takes between 20 seconds (German, CPU) and about 14 minutes (Credit, CPU), or about 1–2 minutes on the GPU. The code, configurations and the raw results of every run, including per-round logs, will be made publicly available upon publication.

7. Results

7.1. Fairness and utility

Table 2 reports test AUC and DPD for all methods on the eight datasets; the full table with hard DPD, EOD, backbones, the number of seeds within the budget and the record of Proposition 5 is in the appendix (Table A.9). We read it in three groups.

Where the budget binds. On Credit, Pokec-z and Pokec-n the unconstrained clipped model (ClipMean) has a gap above the budget $\tau = 0.03$ (0.071, 0.042 and 0.052). CLAMP brings it to 0.018, 0.026 and 0.028, with AUC 0.76, 0.75 and 0.76 against 0.77, 0.75 and 0.75 for ClipMean; on Credit this AUC difference of 0.01 is statistically significant. The reduction is significant against ClipMean on all three graphs, but on the two Pokec graphs CLAMP's gap is not significantly lower than that of plain FedAvg-GAT (0.031 and 0.034), whose gap is already close to the budget. The test gap is at or below τ for 8, 9 and 5 of the 10 seeds; the remaining seeds exceed it on test data although the returned model met the budget on validation data, which is the validation–test generalisation gap that Proposition 4 does not cover. The

baselines that read per-client fairness or group rates in the clear reach lower gaps: F^2GNN 0.006–0.008 at AUC 0.74–0.76, and FairFed 0.004–0.008, but at a large AUC cost on the Pokec graphs (0.62–0.64). CLAMP is not designed to drive the gap to zero; it holds it at the budget the federation chooses, and Section 7.2 shows that lower budgets are reachable. FairGFL, FedGraph-Fair and FairSIN leave the gap close to that of FedAvg on Credit; FedFACT reduces it on the Pokec graphs (0.024 and 0.012) but at AUC 0.59–0.63. The two private methods, PUFFLE and FedFDP, have small gaps because they barely learn: with gradient noise at $\varepsilon = 8$ their AUC lies between 0.37 and 0.66, and below 0.5 on NBA, i.e. no better than chance. These two baselines are therefore not meaningful competitors at this privacy level in our implementation.

Where the budget does not bind. On Bail and AMiner-S the unconstrained gap is already below τ (0.016 and 0.005), so the multiplier stays at zero, CLAMP coincides with ClipMean, and all reasonable methods are within noise of each other.

Small graphs. On German, NBA and Facebook (403 to 1,045 nodes) every method has a large seed-to-seed spread, and CLAMP improves only modestly on ClipMean (German 0.088 to 0.058, NBA 0.093 to 0.076, Facebook 0.086 to 0.069), and in 2 to 4 of 10 runs no release round after the warm-up was feasible, so the output rule fell back to the last model. With a few dozen validation nodes per group and client, the validation gap is a poor proxy for the test gap; switching the statistics noise off does not change this (Section 7.5).

The output rule. The rows labelled "last iterate" return the final model instead of the stored feasible one. On Credit this changes the mean gap from 0.018 to 0.055 and its standard deviation from 0.015 to 0.046: the signed penalty makes the gap oscillate around the budget, and the last iterate lands anywhere on the oscillation. With ten seeds this difference is not significant after Holm correction, and on the Pokec graphs the last iterate is as good as the selected model (0.027 and 0.028), so the rule matters mainly on Credit, where it reduces the spread. Selection without the controller (CLAMP-Select, $\eta = 0$) does not reproduce the effect: its gap is 0.067, 0.041 and 0.052 on Credit, Pokec-z and Pokec-n, close to ClipMean, and it finds an eligible feasible round in only 7, 2 and 1 of 10 runs. The controller creates feasible models and the output rule picks one. On Credit and the Pokec graphs the

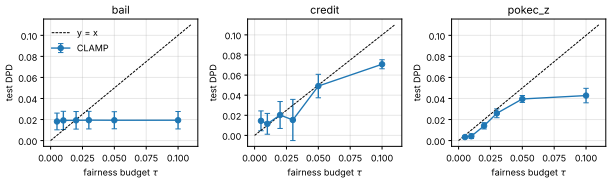


Figure 1: Test DPD of CLAMP against the budget τ (mean over five seeds; dashed: DPD $= \tau$).

selected round is late (98–99 on average), so the returned model has been trained almost as long as the last one.

7.2. Controllability of the fairness budget

Figure 1 varies τ from 0.005 to 0.1. On Pokec-z the test gap follows the budget (0.003, 0.004, 0.014, 0.026, 0.040 and 0.043 for the six values) while AUC stays at 0.74–0.75. On Credit the gap follows the budget down to about 0.012–0.020 and does not go lower: it is 0.012–0.020 for every $\tau \leq 0.03$, rises to 0.049 at $\tau = 0.05$ and reaches the unconstrained 0.071 at $\tau = 0.1$, and the tighter budgets cost about 0.01–0.02 AUC (0.77 to 0.75–0.76). On Bail the gap stays at 0.018–0.019 for every budget, with unchanged AUC: the controller does not reduce the test gap of this dataset. The budget therefore sets the operating point on two of the three datasets, within a range that depends on the dataset. At $\tau = 0.005$, CLAMP on Pokec-z reaches a lower gap than the best baseline (0.003 against 0.008 for F^2GNN) at a similar AUC (0.75 against 0.74).

7.3. Byzantine updates

7.4. Falsified statistics

Table 3 lets $b \in \{1, \ldots, 4\}$ of $K = 10$ clients falsify their statistics. First, the bound of Theorem 3 is not informative here. Computed with the public cap, it ranges from 0.74 to 1.83 and exceeds 1, and is therefore vacuous, in 10 of the 12 attacked settings, while the observed deviation of the estimate from the honest gap was at most 0.092. That it held in all 7,200 attacked release rounds says nothing beyond this. Second, the attack is effective where the budget binds. On Credit a single falsifying client out of ten raises the test gap of CLAMP from 0.011 to 0.064, and on Pokec-z from 0.028 to 0.045. Both values are close to the gaps of the models trained without fairness control in Table 2 (0.071 and 0.042, obtained with $K = 5$), and AUC is unchanged:

the adversary can switch the controller off. With four liars out of ten the swap attack does worse in one Credit run: the corrupted estimate drives the multiplier up, no round after the warm-up is feasible, and the returned last model has a gap of 0.46. A minority that controls the statistics can therefore not only disable fairness control but, through a runaway multiplier, damage the model it is meant to protect. The "mid" attack, which reports the honest pooled rate for both groups, is as effective as the swap attack. On Bail the attack has no effect because the controller is inactive there. Protecting the fairness channel against a small number of liars without seeing individual reports is the main open problem this work leaves (Section 8).

7.5. Privacy

Empirical attack on the statistics. Table 4 runs the strongest attack against one client's released statistics: the adversary knows every validation prediction and every other node's attribute and runs the likelihood-ratio test for the target's attribute. Its balanced accuracy stays far below the bound $(e^{\varepsilon} + \delta)/(1 + e^{\varepsilon})$ implied by Theorem 1: at $\varepsilon = 8$ it is 0.52 (Bail) and 0.58 (German) for one release and 0.73–0.75 when the adversary combines all releases of an unchanged model, which is the worst case the theorem accounts for. Without noise the attack is exact.

Privacy budget and release period. Table 5 varies the statistics budget ε and the release period P on Credit, where the budget binds (five seeds). With per-round releases the controller keeps the gap at the level of the noise-free protocol for $\varepsilon \geq 8$ (0.012 at $\varepsilon = 8$, 0.023 at $\varepsilon = 16$, 0.028 without noise; the differences among these three are within seed noise), while at $\varepsilon \leq 4$ the gap rises to 0.045–0.065 with large seed-to-seed spread: the estimate is too noisy and the multiplier reacts to noise. At $\varepsilon = 1$, releasing every five rounds helps (0.065 to 0.018), because the budget is split over fewer releases. Releasing every ten rounds does not help: the controller then receives only ten updates in 100 rounds and is too slow to bring the gap below the budget. At $\varepsilon \geq 8$ no eligible release round is then feasible in 5 of 5 runs, so the output rule falls back to the last model (gap 0.036–0.040). On a graph of this size per-round releases with ε around 8 work best. On Bail with a tight budget ($\tau = 0.005$, Table 6), Clamp barely lowers the test gap even without statistics noise (0.015 against 0.019 for ClipMean), so this dataset says little about the privacy cost of control. On German (Table A.10 in the appendix) the results are noisy, and per-round releases are not worse than

$P = 10$ for most budgets, contrary to the expectation behind our choice of $P = 10$ for the small graphs. Nor is the noise the main limitation on the small graphs: on NBA, switching the statistics noise off gives a gap of 0.065 $\pm$ 0.057, against 0.069 $\pm$ 0.063 with $\varepsilon = 8$ and $P = 10$, and on Facebook it gives 0.085. With a few dozen validation nodes per group and client, the validation gap is too poor a proxy for the test gap for control to work, with or without privacy.

What the model still reveals. Theorem 1 does not cover the model. Table 7 trains a logistic-regression probe to predict the sensitive attribute from the final model's hidden representations, and compares it with the same probe on the raw node features and on an untrained GAT. On German the probe reaches AUC 0.86–0.87 for FedAvg, ClipMean and Clamp alike, but the raw features already reach 0.79–0.90 and the untrained GAT 0.88–0.94; on Bail the trained models reach 0.54, below the raw features (0.59–0.62). The attribute is therefore predictable from the data itself, and training, with or without demographic-parity control, does not make the representation more revealing than the input. Demographic-parity control does not remove the attribute either, and Clamp makes no claim that it does.

7.6. Ablations

Table 8 varies one protocol constant at a time around the defaults ($C = 1$, $\eta = 0.5$, $\sigma_u = 0$) on Bail and Credit, with three seeds per configuration, so differences below about 0.01 in DPD are within seed noise. On Bail the gap is 0.019–0.024 for every configuration: the budget does not bind there and no constant matters. On Credit, all configurations except the large step $\eta = 2$ and the loose clip $C = 2$ keep the gap at or below the budget on average. A larger step makes the multiplier overshoot and the gap oscillate more widely (0.031 $\pm$ 0.021), and a looser clip lets each local round move the gap further, with no gain in AUC (0.75 against 0.76). Adding Gaussian noise to the update sum ($\sigma_u = 0.05$; far too little noise for a meaningful client-level privacy level) leaves AUC unchanged and makes the gap noisier (0.029 $\pm$ 0.016). The default configuration on Credit has a gap of 0.005 $\pm$ 0.003 here, lower than in the ten-seed main comparison (0.018), which illustrates the seed variance; the proportional direction (Clamp-P, 0.007) and the variants with $\sigma_u = 0.01$ (0.020) and the last iterate (0.020 $\pm$ 0.027) are within this variance.

8. Discussion and limitations

What the sums-only restriction costs. CLAMP gives up two things to stay compatible with secure aggregation. First, its robustness is a norm bound, not a filter: an adversary that stays within the bound still shifts the average by up to $2fC/(K-f)$ per round (Theorem 2), and rules that inspect individual updates can do better against attacks whose updates look unlike honest ones. Second, clipping at a fixed C slows honest training when honest updates are larger than C, which is the main source of the utility gap between CLAMP and unconstrained FedAvg (compare CLAMP-ClipMean with FedAvg-GAT in Table 2). An adaptive, privately estimated bound would reduce this cost, at the price of another channel to account for and protect.

Small graphs. The fairness controller is only as good as its gap estimate, and on small graphs the validation estimate is a poor proxy for the test gap. On German, the 250 validation nodes are split over five clients, so a client typically holds 15 to 35 validation nodes per group. Statistics noise adds to this, but it is not the main problem: switching the noise off does not make control effective on NBA or Facebook. Larger or pooled validation sets, or a fairness criterion with lower variance, would be needed. Theorem 3 makes the same point quantitatively: its bound is weak when honest validation mass is small relative to fn_{cap}.

What is and is not private. Theorem 1 is an attribute-level guarantee for the statistics channel only. The trained model, and therefore the aggregate update, still depends on the training nodes' attributes through the fairness penalty, and the probe in Section 7.5 shows how much of that dependence is measurable. Making the whole pipeline differentially private at the client level would need noise on the updates, which we evaluate only as an ablation.

Simulation of the cryptographic layer. We simulate verifiable secure aggregation in plaintext. The guarantees assume a deployment in which norm and range proofs are enforced cryptographically and enough honest clients participate for secure aggregation to hide individual inputs. We do not measure the cost of that layer; systems such as RoFL [30] report it.

Scope of the fairness notion. We control one binary sensitive attribute and the demographic-parity gap. Equal opportunity needs label-conditioned statistics, which fit the same template (four more sums per release; the ℓ_2 sensitivity grows from 2 to $2\sqrt{2}$ under the attribute-neighbouring relation), but

we have not evaluated it. Multi-valued attributes need one multiplier per constraint.

9. Conclusion

We asked what a federation can achieve for group fairness, privacy and robustness if its server only ever sees validated sums. CLAMP answers with three ingredients: clipped secure aggregation of updates, four privatised group statistics per release computed without reading the sensitive attribute, and a single broadcast Lagrange multiplier, completed by an output rule that returns a model whose released gap met the budget. Each ingredient comes with a result whose scope is stated: attribute-level differential privacy for validation nodes in the statistics channel, a per-round bound on Byzantine influence on the model, a bound on the influence of falsified statistics on the fairness estimate, a fairness guarantee for the returned model on honest validation data, and an a-posteriori record of the average estimated constraint violation. The two Byzantine bounds are loose, and the second is vacuous in most of the settings we evaluate.

On the three graphs where the budget binds (Credit, Pokec-z and Pokec-n), CLAMP brings the test gap to or near the budget at an AUC within 0.01 of an unconstrained clipped model, and the budget is a usable control knob within a dataset-dependent range; on Bail it has no effect, and on the small graphs only a modest one. The experiments also show the limits of the sums-only design. Methods that read per-client fairness information in the clear reach lower gaps; on graphs with a few hundred nodes the privatised statistics are too noisy for effective control; demographic-parity control does not remove the sensitive attribute from the learned representation; and a single client that falsifies its statistics can switch the controller off, and with four liars out of ten it can, in some runs, drive the multiplier to a level that damages the model. Robust aggregation of the fairness statistics that works on sums alone is the most important direction this work leaves open.

CRediT authorship contribution statement

Ngoc-Son-An Nguyen: Software, Investigation, Validation, Writing – original draft. **Quang-Vinh Dang:** Conceptualization, Methodology, Formal analysis, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All eight datasets are public benchmarks (German, Bail, Credit, Pokec-z, Pokec-n, NBA, Facebook and AMiner-S). The code, configuration files and raw per-run results, including per-round logs, will be made publicly available upon publication.

Appendix A. Supplementary results

Table A.9 extends Table 2 with the backbone of each method, the hard demographic-parity gap at threshold 0.5, the equal-opportunity gap, the record $\lambda^{(R)}/(\eta R)$ of Proposition 5 for the CLAMP variants, and the number of seeds whose test gap is within the budget. Table A.10 reports the privacy-budget and release-period sweep on German, Bail, NBA and Facebook.

Table A.9: Full main comparison, mean±sd over seed
for all datasets and methods. DPD (soft) is th
demographic-parity gap of the scores; DPD (hard) uses
0.5 threshold. Cert.: $\lambda^{R}/(\eta R)$ for the CLAMP variant
Backbone: GCN or GAT as in the method definitio
own: the baseline's own architecture. Degenerate ru
(AUC near chance) are included.

Dataset	Method	Backbone	AUC	DPD (soft)	DPD (
German	FedAvg-GCN	GCN	0.74±0.05	0.087±0.077	0.092±
German	FedAvg-GAT	GAT	0.76±0.05	0.110±0.058	0.110±
German	FairFed	GCN	0.75±0.05	0.049±0.039	0.077±
German	FairGNN	own	0.57±0.06	0.111±0.082	0.114±
German	FairSIN	own	0.72±0.05	0.092±0.062	0.099±
German	F2GNN	GAT	0.73±0.02	0.042±0.030	0.065±

Dataset	Method	Backbone	AUC	DPD (soft)	DPD (
German	FairGFL	GCN	0.74±0.05	0.081±0.070	0.096±
German	FedGraph-Fair	GCN	0.66±0.09	0.080±0.051	0.076±
German	PUFFLE	GCN	0.49±0.08	0.011±0.007	0.082±
German	FedFACT	GCN	0.72±0.04	0.078±0.064	0.071±
German	FedFDP	GCN	0.53±0.09	0.014±0.021	0.071±
German	ClipMean (no controller, last iterate)	GAT	0.73±0.06	0.088±0.069	0.110±
German	CLAMP (last iterate)	GAT	0.73±0.07	0.050±0.046	0.052±
German	CLAMP-GCN (last iterate)	GCN	0.73±0.05	0.041±0.044	0.058±
German	CLAMP	GAT	0.72±0.06	0.058±0.044	0.073±
German	CLAMP-GCN	GCN	0.73±0.05	0.065±0.044	0.097±
German	CLAMP-Select (no controller)	GAT	0.73±0.06	0.077±0.066	0.094±
Bail	FedAvg-GCN	GCN	0.80±0.03	0.020±0.011	0.032±
Bail	FedAvg-GAT	GAT	0.80±0.04	0.019±0.011	0.023±
Bail	FairFed	GCN	0.80±0.03	0.012±0.007	0.020±
Bail	FairGNN	own	0.56±0.09	0.019±0.020	0.021±
Bail	FairSIN	own	0.79±0.03	0.025±0.008	0.033±
Bail	F2GNN	GAT	0.78±0.03	0.012±0.008	0.019±
Bail	FairGFL	GCN	0.78±0.03	0.015±0.006	0.021±
Bail	FedGraph-Fair	GCN	0.76±0.03	0.010±0.007	0.018±
Bail	PUFFLE	GCN	0.50±0.06	0.003±0.003	0.010±
Bail	FedFACT	GCN	0.80±0.03	0.013±0.008	0.011±
Bail	FedFDP	GCN	0.51±0.03	0.004±0.003	0.019±
Bail	ClipMean (no controller, last iterate)	GAT	0.78±0.03	0.016±0.009	0.020±
Bail	CLAMP (last iterate)	GAT	0.78±0.03	0.016±0.009	0.020±
Bail	CLAMP-GCN (last iterate)	GCN	0.79±0.02	0.014±0.006	0.022±
Bail	CLAMP	GAT	0.78±0.03	0.016±0.009	0.021±
Bail	CLAMP-GCN	GCN	0.79±0.02	0.015±0.006	0.022±
Bail	CLAMP-Select (no controller)	GAT	0.78±0.03	0.016±0.009	0.021±
Credit	FedAvg-GCN	GCN	0.76±0.01	0.072±0.010	0.110±
Credit	FedAvg-GAT	GAT	0.77±0.01	0.078±0.010	0.116±
Credit	FairFed	GCN	0.75±0.01	0.008±0.005	0.048±
Credit	FairGNN	own	0.67±0.09	0.100±0.055	0.099±
Credit	FairSIN	own	0.75±0.00	0.070±0.012	0.105±
Credit	F2GNN	GAT	0.76±0.01	0.006±0.007	0.063±
Credit	FairGFL	GCN	0.76±0.01	0.072±0.009	0.108±

Dataset	Method	Backbone	AUC	DPD (soft)	DPD (
Credit	FedGraph-Fair	GCN	0.75±0.01	0.075±0.014	0.100±
Credit	PUFFLE	GCN	0.46±0.13	0.016±0.008	0.070±
Credit	FedFACT	GCN	0.75±0.01	0.070±0.016	0.045±
Credit	FedFDP	GCN	0.54±0.11	0.017±0.009	0.073±
Credit	ClipMean (no controller, last iterate)	GAT	0.77±0.01	0.071±0.009	0.104±
Credit	CLAMP (last iterate)	GAT	0.76±0.01	0.055±0.046	0.098±
Credit	CLAMP-GCN (last iterate)	GCN	0.75±0.01	0.046±0.031	0.080±
Credit	CLAMP	GAT	0.76±0.01	0.018±0.015	0.056±
Credit	CLAMP-GCN	GCN	0.75±0.01	0.014±0.013	0.036±
Credit	CLAMP-Select (no controller)	GAT	0.77±0.01	0.067±0.010	0.097±
Pokec-z	FedAvg-GCN	GCN	0.62±0.07	0.038±0.010	0.037±
Pokec-z	FedAvg-GAT	GAT	0.74±0.02	0.031±0.013	0.036±
Pokec-z	FairFed	GCN	0.64±0.05	0.004±0.003	0.010±
Pokec-z	FairGNN	own	0.54±0.08	0.005±0.013	0.005±
Pokec-z	FairSIN	own	0.66±0.05	0.026±0.010	0.038±
Pokec-z	F2GNN	GAT	0.74±0.02	0.008±0.007	0.012±
Pokec-z	FairGFL	GCN	0.66±0.02	0.041±0.011	0.043±
Pokec-z	FedGraph-Fair	GCN	0.64±0.03	0.039±0.013	0.041±
Pokec-z	PUFFLE	GCN	0.55±0.11	0.008±0.009	0.022±
Pokec-z	FedFACT	GCN	0.63±0.04	0.024±0.014	0.034±
Pokec-z	FedFDP	GCN	0.64±0.11	0.008±0.008	0.029±
Pokec-z	ClipMean (no controller, last iterate)	GAT	0.75±0.02	0.042±0.011	0.055±
Pokec-z	CLAMP (last iterate)	GAT	0.75±0.02	0.027±0.004	0.028±
Pokec-z	CLAMP-GCN (last iterate)	GCN	0.72±0.01	0.028±0.006	0.025±
Pokec-z	CLAMP	GAT	0.75±0.02	0.026±0.004	0.025±
Pokec-z	CLAMP-GCN	GCN	0.72±0.02	0.028±0.006	0.027±
Pokec-z	CLAMP-Select (no controller)	GAT	0.75±0.02	0.041±0.010	0.069±
Pokec-n	FedAvg-GCN	GCN	0.62±0.06	0.031±0.018	0.025±
Pokec-n	FedAvg-GAT	GAT	0.75±0.02	0.034±0.017	0.037±
Pokec-n	FairFed	GCN	0.62±0.06	0.006±0.004	0.007±
Pokec-n	FairGNN	own	0.50±0.00	0.000±0.000	0.000±
Pokec-n	FairSIN	own	0.64±0.06	0.043±0.014	0.049±
Pokec-n	F2GNN	GAT	0.74±0.03	0.008±0.007	0.006±
Pokec-n	FairGFL	GCN	0.65±0.03	0.044±0.007	0.042±
Pokec-n	FedGraph-Fair	GCN	0.59±0.05	0.024±0.015	0.018±

Dataset	Method	Backbone	AUC	DPD (soft)	DPD (
Pokec-n	PUFFLE	GCN	0.57±0.11	0.010±0.006	0.038±
Pokec-n	FedFACT	GCN	0.59±0.06	0.012±0.011	0.016±
Pokec-n	FedFDP	GCN	0.66±0.08	0.010±0.008	0.026±
Pokec-n	ClipMean (no controller, last iterate)	GAT	0.75±0.02	0.052±0.016	0.064±
Pokec-n	CLAMP (last iterate)	GAT	0.75±0.02	0.028±0.012	0.027±
Pokec-n	CLAMP-GCN (last iterate)	GCN	0.71±0.02	0.028±0.009	0.026±
Pokec-n	CLAMP	GAT	0.76±0.02	0.028±0.012	0.028±
Pokec-n	CLAMP-GCN	GCN	0.71±0.02	0.027±0.008	0.026±
Pokec-n	CLAMP-Select (no controller)	GAT	0.75±0.02	0.052±0.015	0.090±
AMiner-S	FedAvg-GCN	GCN	0.93±0.00	0.005±0.003	0.005±
AMiner-S	FedAvg-GAT	GAT	0.94±0.00	0.007±0.004	0.007±
AMiner-S	FairFed	GCN	0.94±0.00	0.005±0.003	0.006±
AMiner-S	FairGNN	own	0.63±0.08	0.098±0.035	0.072±
AMiner-S	FairSIN	own	0.92±0.01	0.005±0.003	0.005±
AMiner-S	F2GNN	GAT	0.94±0.01	0.005±0.004	0.006±
AMiner-S	FairGFL	GCN	0.94±0.01	0.003±0.002	0.003±
AMiner-S	FedGraph-Fair	GCN	0.93±0.01	0.003±0.003	0.004±
AMiner-S	PUFFLE	GCN	0.49±0.03	0.006±0.005	0.008±
AMiner-S	FedFACT	GCN	0.92±0.01	0.008±0.004	0.004±
AMiner-S	FedFDP	GCN	0.49±0.03	0.006±0.004	0.011±
AMiner-S	ClipMean (no controller, last iterate)	GAT	0.94±0.01	0.005±0.004	0.004±
AMiner-S	CLAMP (last iterate)	GAT	0.94±0.01	0.005±0.003	0.004±
AMiner-S	CLAMP-GCN (last iterate)	GCN	0.94±0.00	0.002±0.002	0.003±
AMiner-S	CLAMP	GAT	0.94±0.01	0.004±0.004	0.004±
AMiner-S	CLAMP-GCN	GCN	0.94±0.00	0.002±0.002	0.003±
AMiner-S	CLAMP-Select (no controller)	GAT	0.94±0.01	0.005±0.003	0.004±
NBA	FedAvg-GCN	GCN	0.78±0.04	0.084±0.047	0.085±
NBA	FedAvg-GAT	GAT	0.81±0.06	0.099±0.059	0.102±
NBA	FairFed	GCN	0.80±0.05	0.081±0.062	0.100±
NBA	FairGNN	own	0.77±0.08	0.097±0.044	0.100±
NBA	FairSIN	own	0.73±0.08	0.087±0.059	0.083±
NBA	F2GNN	GAT	0.78±0.13	0.109±0.072	0.148±
NBA	FairGFL	GCN	0.76±0.04	0.078±0.044	0.081±
NBA	FedGraph-Fair	GCN	0.77±0.10	0.084±0.043	0.089±
NBA	PUFFLE	GCN	0.37±0.19	0.014±0.012	0.061±

Dataset	Method	Backbone	AUC	DPD (soft)	DPD (
NBA	FedFACT	GCN	0.76±0.04	0.142±0.082	0.112±
NBA	FedFDP	GCN	0.39±0.20	0.015±0.010	0.058±
NBA	ClipMean (no controller, last iterate)	GAT	0.78±0.08	0.093±0.068	0.107±
NBA	CLAMP (last iterate)	GAT	0.79±0.07	0.092±0.068	0.106±
NBA	CLAMP-GCN (last iterate)	GCN	0.77±0.07	0.070±0.046	0.076±
NBA	CLAMP	GAT	0.80±0.08	0.076±0.054	0.081±
NBA	CLAMP-GCN	GCN	0.78±0.07	0.077±0.044	0.069±
NBA	CLAMP-Select (no controller)	GAT	0.79±0.09	0.061±0.051	0.084±
Facebook	FedAvg-GCN	GCN	0.76±0.03	0.074±0.036	0.079±
Facebook	FedAvg-GAT	GAT	0.77±0.05	0.078±0.048	0.082±
Facebook	FairFed	GCN	0.76±0.03	0.049±0.031	0.050±
Facebook	FairGNN	own	0.51±0.05	0.024±0.041	0.028±
Facebook	FairSIN	own	0.74±0.05	0.070±0.033	0.075±
Facebook	F2GNN	GAT	0.76±0.05	0.069±0.040	0.075±
Facebook	FairGFL	GCN	0.75±0.05	0.080±0.040	0.081±
Facebook	FedGraph-Fair	GCN	0.74±0.05	0.094±0.037	0.091±
Facebook	PUFFLE	GCN	0.53±0.10	0.009±0.006	0.042±
Facebook	FedFACT	GCN	0.74±0.05	0.055±0.042	0.069±
Facebook	FedFDP	GCN	0.54±0.08	0.013±0.009	0.044±
Facebook	ClipMean (no controller, last iterate)	GAT	0.77±0.06	0.086±0.043	0.086±
Facebook	CLAMP (last iterate)	GAT	0.77±0.06	0.082±0.043	0.082±
Facebook	CLAMP-GCN (last iterate)	GCN	0.76±0.05	0.071±0.031	0.081±
Facebook	CLAMP	GAT	0.76±0.06	0.069±0.034	0.080±
Facebook	CLAMP-GCN	GCN	0.76±0.04	0.078±0.030	0.089±
Facebook	CLAMP-Select (no controller)	GAT	0.76±0.06	0.082±0.032	0.096±

References

[1] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations (ICLR), 2017.
URL https://openreview.net/forum?id=SJU4ayYgl

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning

Representations (ICLR), 2018.
URL https://openreview.net/forum?id=rJXMpikCZ

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics (AISTATS), Vol. 54, 2017, pp. 1273–1282.
URL https://proceedings.mlr.press/v54/mcmahan17a.html

[4] E. Dai, S. Wang, Say no to the discrimination: Learning fair graph neural networks with limited sensitive attribute information, in: ACM International Conference on Web Search and Data Mining (WSDM), 2021, pp. 680–688. doi:10.1145/3437963.3441752.

[5] C. Agarwal, H. Lakkaraju, M. Zitnik, Towards a unified framework for fair and stable graph representation learning, in: Uncertainty in Artificial Intelligence (UAI), PMLR, 2021, pp. 2114–2124.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016, pp. 308–318. doi:10.1145/2976749.2978318.

[7] H. B. McMahan, D. Ramage, K. Talwar, L. Zhang, Learning differentially private recurrent language models, in: International Conference on Learning Representations (ICLR), 2018.
URL https://openreview.net/forum?id=BJ0hF1Z0b

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, J. Stainer, Machine learning with adversaries: Byzantine tolerant gradient descent, in: Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 119–129.
URL https://proceedings.neurips.cc/paper/2017/hash/f4b9ec30ad9f68f89b29639786cb62ef-Abstract.html

[9] D. Yin, Y. Chen, K. Ramchandran, P. L. Bartlett, Byzantine-robust distributed learning: Towards optimal statistical rates, in: International Conference on Machine Learning (ICML), Vol. 80, 2018.
URL https://proceedings.mlr.press/v80/yin18a.html

[10] G. Baruch, M. Baruch, Y. Goldberg, A little is enough: Circumventing defenses for distributed learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8632–8642.
URL https://proceedings.neurips.cc/paper/2019/hash/ec1c59141046cd1866bbbcdfb6ae31d4-Abstract.html

[11] Y. H. Ezzeldin, S. Yan, C. He, E. Ferrara, A. S. Avestimehr, FairFed: Enabling group fairness in federated learning, in: AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 7494–7502. doi:10.1609/aaai.v37i6.25911.

[12] T. Li, M. Sanjabi, A. Beirami, V. Smith, Fair resource allocation in federated learning, in: International Conference on Learning Representations (ICLR), 2020.
URL https://openreview.net/forum?id=ByexElSYDr

[13] L. Zhang, Z. Han, X. Feng, J. Zhang, Y. Li, C. Chen, FedFACT: A provable framework for controllable group-fairness calibration in federated learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2025.
URL https://arxiv.org/abs/2506.03777

[14] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.

[15] L. Corbucci, M. A. Heikkilä, D. Solans Noguero, A. Monreale, N. Kourtellis, PUFFLE: Balancing privacy, utility, and fairness in federated learning, in: European Conference on Artificial Intelligence (ECAI), Vol. 392 of Frontiers in Artificial Intelligence and Applications, 2024, pp. 1639–1647. doi:10.3233/FAIA240671.

[16] X. Ling, J. Fu, K. Wang, H. Li, T. Cheng, Z. Chen, FedFDP: Fairness-aware federated learning with differential privacy, arXiv preprint arXiv:2402.16028 (2024).
URL https://arxiv.org/abs/2402.16028

[17] Q.-V. Dang, N.-S.-A. Nguyen, FedFairGNN: A privacy-preserving and fairness-aware federated graph neural network for fraud detection, in: Proceedings of IndabaX Nigeria 2026: Building Scalable AI That Works: From Research to Deployment in Resource-Constrained Environments, Vol. 319 of Proceedings of Machine Learning Research, 2026, pp. 74–86. URL https://proceedings.mlr.press/v319/dang26a.html

[18] Y. Dong, N. Liu, B. Jalaian, J. Li, EDITS: Modeling and mitigating data bias for graph neural networks, in: The Web Conference (WWW), 2022, pp. 1259–1269. doi:10.1145/3485447.3512173.

[19] C. Yang, J. Liu, Y. Yan, C. Shi, FairSIN: Achieving fairness in graph neural networks through sensitive information neutralization, in: AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 9241–9249. doi:10.1609/aaai.v38i8.28776.

[20] Z. Li, Y. Dong, Q. Liu, J. X. Yu, Rethinking fair graph neural networks from re-balancing, in: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024, pp. 1736–1745. doi:10.1145/3637528.3671826.

[21] Y. Zhu, J. Li, Y. Bian, Z. Zheng, L. Chen, One fits all: Learning fair graph neural networks for various sensitive attributes, in: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024. doi:10.1145/3637528.3672029.

[22] N. Cui, X. Wang, W. H. Wang, V. Chen, Y. Ning, Equipping federated graph neural networks with structure-aware group fairness, in: IEEE International Conference on Data Mining (ICDM), 2023. doi:10.1109/ICDM58522.2023.00111.

[23] Z. Zhou, S. Yang, F. Zhao, X. Ren, FairGFL: Privacy-preserving fairness-aware federated learning with overlapping subgraphs, IEEE Transactions on Parallel and Distributed Systems 37 (3) (2026). doi:10.1109/TPDS.2025.3649863.

[24] K. Khan, FedGraph-Fair: Federated learning with personalization and fairness via dynamic graphs and distributionally robust optimization, Information Sciences 728 (2026) 122710. doi:10.1016/j.ins.2025.122710.

[25] A. Cotter, H. Jiang, M. Gupta, S. Wang, T. Narayan, S. You, K. Sridharan, Optimization with non-differentiable constraints with applications to fairness, recall, churn, and other goals, Journal of Machine Learning Research 20 (172) (2019) 1–59.

[26] E. M. El Mhamdi, R. Guerraoui, S. Rouault, The hidden vulnerability of distributed learning in byzantium, in: International Conference on Machine Learning (ICML), Vol. 80, 2018, pp. 3521–3530.
URL https://proceedings.mlr.press/v80/mhamdi18a.html

[27] X. Cao, M. Fang, J. Liu, N. Z. Gong, FLTrust: Byzantine-robust federated learning via trust bootstrapping, in: Network and Distributed System Security Symposium (NDSS), 2021. doi:10.14722/ndss.2021.24434.

[28] C. Xie, O. Koyejo, I. Gupta, Fall of empires: Breaking byzantine-tolerant SGD by inner product manipulation, in: Uncertainty in Artificial Intelligence (UAI), Vol. 115, 2019, pp. 261–270.
URL https://proceedings.mlr.press/v115/xie20a.html

[29] H. Corrigan-Gibbs, D. Boneh, Prio: Private, robust, and scalable computation of aggregate statistics, in: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 259–282.

[30] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, A. Hithnawi, RoFL: Attestable robustness for secure federated learning, arXiv preprint arXiv:2107.03311 (2023).
URL https://arxiv.org/abs/2107.03311

[31] P. Kairouz, Z. Liu, T. Steinke, The distributed discrete Gaussian mechanism for federated learning with secure aggregation, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 5201–5212.

[32] A.-A. Bendoukha, D. Demirag, N. Kaaniche, A. Boudguiga, R. Sirdey, S. Gambs, Towards privacy-preserving and fairness-aware federated learning framework, Proceedings on Privacy Enhancing Technologies (PoPETs) 2025 (1) (2025) 845–865. doi:10.56553/popets-2025-0044.

[33] J. So, B. Güler, A. S. Avestimehr, Byzantine-resilient secure federated learning, IEEE Journal on Selected Areas in Communications (2021). doi:10.1109/JSAC.2020.3041404.

[34] M. Rathee, C. Shen, S. Wagh, R. A. Popa, ELSA: Secure aggregation for federated learning with malicious actors, in: 2023 IEEE Symposium on Security and Privacy (SP), 2023. doi:10.1109/SP46215.2023.10179468.

[35] I. Mironov, Rényi differential privacy, in: IEEE Computer Security Foundations Symposium (CSF), 2017, pp. 263–275. doi:10.1109/CSF.2017.11.

[36] C. L. Canonne, G. Kamath, T. Steinke, The discrete Gaussian for differential privacy, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 15676–15688.

[37] J. McAuley, J. Leskovec, Learning to discover social circles in ego networks, in: Advances in Neural Information Processing Systems (NIPS), Vol. 25, 2012.

[38] Y. Dong, Z. Lei, Z. Zheng, S. Wang, J. Ma, A. J. Huang, C. Chen, J. Li, PyGDebias: A Python library for debiasing in graph learning, in: Companion Proceedings of the ACM Web Conference 2024 (WWW '24 Companion), 2024, pp. 1019–1022. doi:10.1145/3589335.3651239.

Table 2: Main results: test AUC and DPD, mean±sd over seeds. Best per column in bold; methods with mean AUC below 0.60 on a dataset (degenerate) are not bolded. †: Holm-adjusted Wilcoxon signed-rank $p < 0.05$ against CLAMP, paired by seed.

	German		Bail		Credit		Pokec-z	
	AUC	DPD	AUC	DPD	AUC	DPD	AUC	DPD
FedAvg-GAT	**0.76±0.05**	0.110±0.058	0.80±0.04	0.019±0.011	0.77±0.01	0.078±0.010†	0.74±0.02	0.031±0.013
FairFed	0.75±0.05	0.049±0.039	**0.80±0.03**	0.012±0.007	0.75±0.01	0.008±0.005	0.64±0.05†	**0.004±0.003**†
F2GNN	0.73±0.02	**0.042±0.030**	0.78±0.03	0.012±0.008	0.76±0.01	**0.006±0.007**	0.74±0.02	0.008±0.007†
FairGFL	0.74±0.05	0.081±0.070	0.78±0.03	0.015±0.006	0.76±0.01	0.072±0.009†	0.66±0.02†	0.041±0.011
FedGraph-Fair	0.66±0.09	0.080±0.051	0.76±0.03	**0.010±0.007**	0.75±0.01	0.075±0.014†	0.64±0.03†	0.039±0.013
FedFACT	0.72±0.04	0.078±0.064	0.80±0.03	0.013±0.008	0.75±0.01	0.070±0.016†	0.63±0.04†	0.024±0.014
PUFFLE	0.49±0.08†	0.011±0.007†	0.50±0.06†	0.003±0.003	0.46±0.13†	0.016±0.008	0.55±0.11†	0.008±0.009†
FedFDP	0.53±0.09†	0.014±0.021	0.51±0.03†	0.004±0.003†	0.54±0.11†	0.017±0.009	0.64±0.11†	0.008±0.008†
FairGNN	0.57±0.06†	0.111±0.082	0.56±0.09†	0.019±0.020	0.67±0.09†	0.100±0.055	0.54±0.08†	0.005±0.013
FairSIN	0.72±0.05	0.092±0.062	0.79±0.03	0.025±0.008†	0.75±0.00	0.070±0.012†	0.66±0.05†	0.026±0.010
ClipMean (no controller, last iterate)	0.73±0.06	0.088±0.069	0.78±0.03	0.016±0.009	**0.77±0.01**†	0.071±0.009†	0.75±0.02	0.042±0.011†
CLAMP (last iterate)	0.73±0.07	0.050±0.046	0.78±0.03	0.016±0.009	0.76±0.01	0.055±0.046	0.75±0.02	0.027±0.004
CLAMP-Select (no controller)	0.73±0.06	0.077±0.066	0.78±0.03	0.016±0.009	0.77±0.01†	0.067±0.010†	**0.75±0.02**	0.041±0.010†
CLAMP	0.72±0.06	0.058±0.044	0.78±0.03	0.016±0.009	0.76±0.01	0.018±0.015	0.75±0.02	0.026±0.004
CLAMP-GCN	0.73±0.05	0.065±0.044	0.79±0.02	0.015±0.006	0.75±0.01	0.014±0.013	0.72±0.02†	0.028±0.006

	Pokec-n		AMiner-S		NBA		Facebook	
	AUC	DPD	AUC	DPD	AUC	DPD	AUC	DPD
FedAvg-GAT	0.75±0.02	0.034±0.017	0.94±0.00	0.007±0.004†	**0.81±0.06**	0.099±0.059	0.77±0.05	0.078±0.048
FairFed	0.62±0.06†	**0.006±0.004**†	0.94±0.00	0.005±0.003	0.80±0.05	0.081±0.062	0.76±0.03	**0.049±0.031**
F2GNN	0.74±0.03†	0.008±0.007†	**0.94±0.01**†	0.005±0.004	0.78±0.13	0.109±0.072	0.76±0.05	0.069±0.040
FairGFL	0.65±0.03†	0.044±0.007†	0.94±0.01	0.003±0.002	0.76±0.04	0.078±0.044	0.75±0.05	0.080±0.040
FedGraph-Fair	0.59±0.05†	0.024±0.015	0.93±0.01†	0.003±0.003	0.77±0.10	0.084±0.043	0.74±0.05	0.094±0.037
FedFACT	0.59±0.06†	0.012±0.011	0.92±0.01†	0.008±0.004	0.76±0.04	0.142±0.082	0.74±0.05	0.055±0.042
PUFFLE	0.57±0.11†	0.010±0.006	0.49±0.03†	0.006±0.005	0.37±0.19	0.014±0.012	0.53±0.10†	0.009±0.006†
FedFDP	0.66±0.08†	0.010±0.008	0.49±0.03†	0.006±0.004	0.39±0.20†	0.015±0.010†	0.54±0.08†	0.013±0.009
FairGNN	0.50±0.00†	0.000±0.000†	0.63±0.08†	0.098±0.035†	0.77±0.08	0.097±0.044	0.51±0.05†	0.024±0.041
FairSIN	0.64±0.06†	0.043±0.014	0.92±0.01†	0.005±0.003	0.73±0.08	0.087±0.059	0.74±0.05	0.070±0.033
ClipMean (no controller, last iterate)	0.75±0.02	0.052±0.016†	0.94±0.01	0.005±0.004	0.78±0.08	0.093±0.068	0.77±0.06	0.086±0.043
CLAMP (last iterate)	0.75±0.02	0.028±0.012	0.94±0.01	0.005±0.003	0.79±0.07	0.092±0.068	**0.77±0.06**	0.082±0.043
CLAMP-Select (no controller)	0.75±0.02	0.052±0.015†	0.94±0.01	0.005±0.003	0.79±0.09	**0.061±0.051**	0.76±0.06	0.082±0.032
CLAMP	**0.76±0.02**	0.028±0.012	0.94±0.01	0.004±0.004	0.80±0.08	0.076±0.054	0.76±0.06	0.069±0.034
CLAMP-GCN	0.71±0.02†	0.027±0.008	0.94±0.00	**0.002±0.002**	0.78±0.07	0.077±0.044	0.76±0.04	0.078±0.030

Table 3: Statistics attacks. $|\widehat{D} - D^H|_{\max}$: largest deviation of the server's estimate from the honest-only gap over the release rounds; Bound: largest Theorem 3 bound (honest-mass term plus statistics-noise term, $\beta = 0.05$) over the release rounds; Viol.: release rounds with the deviation above the bound. Each row covers 300 release rounds (3 seeds of 100 rounds); f is the number of falsifying clients out of $K = 10$. AUC and DPD: mean±sd over seeds.

Dataset	Attack	f	$\lvert\widehat{D} - D^H\rvert_{\max}$	Bound	Viol.	AUC	DPD
Bail	none	0	0.017	0.070	0	0.75±0.02	0.009±0.003
Bail	swap	1	0.014	0.769	0	0.74±0.02	0.009±0.003
Bail	swap	2	0.015	1.273	0	0.75±0.02	0.010±0.003
Bail	swap	3	0.014	1.526	0	0.75±0.02	0.009±0.003
Bail	swap	4	0.011	1.688	0	0.74±0.02	0.009±0.003
Bail	mid	1	0.014	0.769	0	0.75±0.02	0.009±0.003
Bail	mid	2	0.014	1.273	0	0.74±0.02	0.010±0.003
Bail	mid	3	0.014	1.526	0	0.74±0.02	0.009±0.003
Bail	mid	4	0.011	1.688	0	0.74±0.02	0.009±0.003
Pokec-z	none	0	0.006	0.021	0	0.76±0.00	0.028±0.016
Pokec-z	swap	1	0.028	0.738	0	0.77±0.00	0.045±0.008
Pokec-z	swap	2	0.041	1.219	0	0.76±0.00	0.044±0.008
Pokec-z	swap	3	0.037	1.448	0	0.76±0.00	0.045±0.008
Pokec-z	swap	4	0.031	1.592	0	0.77±0.00	0.045±0.008
Pokec-z	mid	1	0.016	0.738	0	0.76±0.00	0.041±0.007
Pokec-z	mid	2	0.026	1.219	0	0.77±0.00	0.046±0.007
Pokec-z	mid	3	0.031	1.448	0	0.76±0.00	0.045±0.009
Pokec-z	mid	4	0.036	1.592	0	0.77±0.00	0.045±0.008
Credit	none	0	0.051	0.141	0	0.76±0.01	0.011±0.004
Credit	swap	1	0.065	1.115	0	0.76±0.01	0.064±0.008
Credit	swap	2	0.058	1.421	0	0.76±0.01	0.065±0.008
Credit	swap	3	0.047	1.617	0	0.76±0.01	0.064±0.007
Credit	swap	4	0.092	1.830	0	0.72±0.09	0.198±0.228
Credit	mid	1	0.057	1.115	0	0.76±0.01	0.065±0.008
Credit	mid	2	0.059	1.421	0	0.76±0.01	0.064±0.008
Credit	mid	3	0.062	1.617	0	0.76±0.01	0.064±0.008
Credit	mid	4	0.063	1.830	0	0.76±0.00	0.064±0.008

Table 4: Differencing attack on the statistics channel: balanced accuracy of the likelihood-ratio test over random validation targets, mean±sd over seeds. Single: one noisy release. All: the R releases of an unchanged vector ($R = 10$ for German, $R = 100$ for Bail), the worst case the theorem accounts for. Bound: $(e^{\varepsilon} + \delta)/(1 + e^{\varepsilon})$. Monte Carlo and closed-form values agree to within 0.005.

ε	Bound	German single	German all	Bail single	Bail all
1	0.731	0.513±0.003	0.542±0.003	0.504±0.001	0.537±0.001
2	0.881	0.525±0.003	0.579±0.004	0.507±0.001	0.570±0.001
4	0.982	0.547±0.003	0.644±0.004	0.513±0.001	0.629±0.001
8	1.000	0.584±0.004	0.747±0.004	0.524±0.001	0.726±0.001
16	1.000	0.645±0.004	0.878±0.003	0.541±0.001	0.854±0.001
∞	1.000	1.000±0.000	1.000±0.000	1.000±0.000	1.000±0.000

Table 5: Privacy: utility and fairness against the statistics budget ε and release period P. $\varepsilon = \infty$: statistics noise off, default period (P=1). mean±sd over 5 seeds. Fallback: number of seeds with no eligible feasible release after the warm-up, so the last model is reported.

ε	P	Credit AUC	Credit DPD	fallback
1	1	0.74±0.04	0.065±0.067	0
1	5	0.76±0.01	0.018±0.013	0
1	10	0.77±0.01	0.036±0.019	0
2	1	0.75±0.02	0.045±0.033	0
2	5	0.77±0.01	0.038±0.024	0
2	10	0.77±0.01	0.037±0.008	2
4	1	0.76±0.01	0.046±0.040	0
4	5	0.77±0.01	0.033±0.008	0
4	10	0.77±0.01	0.038±0.005	4
8	1	0.76±0.01	0.012±0.012	0
8	5	0.77±0.01	0.030±0.008	1
8	10	0.77±0.01	0.039±0.005	5
16	1	0.77±0.01	0.023±0.012	0
16	5	0.77±0.01	0.032±0.007	1
16	10	0.77±0.01	0.040±0.005	5
∞	–	0.77±0.01	0.028±0.007	0

Table 6: Privacy: utility and fairness against the statistics budget ε and release period P, fairness budget $\tau = 0.005$. $\varepsilon = \infty$: statistics noise off, default period (P=1). mean±sd over 5 seeds. Fallback: number of seeds with no eligible feasible release after the warm-up, so the last model is reported. The ClipMean row is the unconstrained reference (no Lagrangian term).

ε	P	Bail AUC	Bail DPD	fallback
1	1	0.78±0.03	0.015±0.007	0
1	5	0.78±0.03	0.018±0.008	0
1	10	0.77±0.04	0.016±0.007	0
2	1	0.78±0.03	0.018±0.008	0
2	5	0.78±0.03	0.017±0.008	0
2	10	0.77±0.04	0.013±0.005	0
4	1	0.78±0.03	0.019±0.007	0
4	5	0.78±0.03	0.018±0.007	0
4	10	0.77±0.04	0.014±0.007	1
8	1	0.79±0.03	0.018±0.008	0
8	5	0.78±0.03	0.018±0.008	2
8	10	0.77±0.03	0.014±0.008	1
16	1	0.78±0.03	0.018±0.007	0
16	5	0.78±0.03	0.018±0.008	2
16	10	0.78±0.03	0.016±0.008	2
∞	–	0.78±0.03	0.015±0.007	2
ClipMean	–	0.78±0.03	0.019±0.009	–

Table 7: Attribute probe on the final global model: AUC of predicting the sensitive attribute from penultimate test-node embeddings, mean±sd over seeds.

Method	German AUC	Bail AUC
FedAvg-GAT	0.87±0.05	0.54±0.01
ClipMean (no controller, last iterate)	0.86±0.06	0.54±0.01
CLAMP (last iterate)	0.86±0.06	0.54±0.01
Control: raw features	0.85±0.05	0.61±0.02
Control: untrained GAT	0.91±0.03	0.55±0.01

Table 8: CLAMP ablations, one factor at a time around $C = 1$, $\eta = 0.5$, $\sigma_u = 0$, mean±sd over seeds. The last two rows at the default setting are the output-rule (last iterate) and direction-rule (proportional) ablations.

Dataset	Method	C	η	σ_u	AUC	DPD
Bail	CLAMP (last iterate)	1	0.5	0	0.78±0.05	0.023±0.004
Bail	CLAMP	0.5	0.5	0	0.78±0.04	0.019±0.004
Bail	CLAMP	1	0.1	0	0.78±0.04	0.023±0.004
Bail	CLAMP	1	0.5	0	0.78±0.04	0.023±0.004
Bail	CLAMP	1	0.5	0.01	0.78±0.05	0.024±0.004
Bail	CLAMP	1	0.5	0.05	0.78±0.05	0.022±0.004
Bail	CLAMP	1	2	0	0.78±0.05	0.023±0.004
Bail	CLAMP	2	0.5	0	0.78±0.04	0.024±0.004
Bail	CLAMP-P (proportional direction)	1	0.5	0	0.78±0.05	0.024±0.003
Credit	CLAMP (last iterate)	1	0.5	0	0.76±0.01	0.020±0.027
Credit	CLAMP	0.5	0.5	0	0.76±0.01	0.019±0.011
Credit	CLAMP	1	0.1	0	0.76±0.01	0.030±0.005
Credit	CLAMP	1	0.5	0	0.76±0.01	0.005±0.003
Credit	CLAMP	1	0.5	0.01	0.76±0.01	0.020±0.020
Credit	CLAMP	1	0.5	0.05	0.76±0.01	0.029±0.016
Credit	CLAMP	1	2	0	0.75±0.01	0.031±0.021
Credit	CLAMP	2	0.5	0	0.75±0.01	0.032±0.016
Credit	CLAMP-P (proportional direction)	1	0.5	0	0.76±0.01	0.007±0.004

Table A.10: Privacy: utility and fairness against the statistics budget ε and release period P. $\varepsilon = \infty$: statistics noise off, default period (P=10 for German, NBA and Facebook, P=1 for Bail). mean±sd over 5 seeds. Fallback: number of seeds with no eligible feasible release after the warm-up, so the last model is reported.

ε	P	German AUC	German DPD	fallback	Bail AUC	Bail DPD	fallback
1	1	0.73±0.08	0.052±0.034	0	0.78±0.03	0.019±0.008	0
1	5	0.65±0.07	0.163±0.099	0	0.78±0.03	0.019±0.008	0
1	10	0.66±0.07	0.244±0.160	2	0.78±0.03	0.019±0.008	0
2	1	0.71±0.06	0.090±0.079	0	0.79±0.03	0.019±0.008	0
2	5	0.69±0.04	0.212±0.141	3	0.78±0.03	0.019±0.008	0
2	10	0.74±0.08	0.144±0.026	1	0.78±0.03	0.019±0.008	0
4	1	0.70±0.05	0.046±0.058	0	0.79±0.03	0.019±0.008	0
4	5	0.72±0.08	0.113±0.072	0	0.78±0.03	0.019±0.007	0
4	10	0.73±0.08	0.102±0.042	1	0.78±0.03	0.019±0.008	0
8	1	0.71±0.06	0.124±0.121	0	0.78±0.03	0.020±0.008	0
8	5	0.73±0.08	0.068±0.020	0	0.78±0.03	0.019±0.007	0
8	10	0.73±0.09	0.081±0.049	1	0.78±0.03	0.019±0.008	0
16	1	0.73±0.04	0.059±0.055	0	0.79±0.03	0.020±0.008	0
16	5	0.73±0.08	0.064±0.036	0	0.78±0.03	0.019±0.007	0
16	10	0.73±0.09	0.069±0.036	2	0.78±0.03	0.019±0.008	0
∞	–	0.73±0.08	0.058±0.040	3	0.79±0.03	0.020±0.008	0

ε	P	NBA AUC	NBA DPD	fallback	Facebook AUC	Facebook DPD	fallback
1	1	0.79±0.10	0.065±0.063	0	0.72±0.04	0.058±0.036	0
1	5	0.82±0.07	0.072±0.057	0	0.71±0.05	0.108±0.071	2
1	10	0.74±0.14	0.077±0.069	2	0.73±0.06	0.108±0.082	3
2	1	0.79±0.11	0.096±0.018	0	0.72±0.05	0.060±0.050	0
2	5	0.74±0.06	0.103±0.041	0	0.74±0.06	0.110±0.041	2
2	10	0.74±0.10	0.121±0.087	3	0.73±0.06	0.112±0.044	3
4	1	0.76±0.12	0.101±0.056	0	0.74±0.05	0.087±0.045	0
4	5	0.77±0.10	0.111±0.077	3	0.74±0.06	0.077±0.055	0
4	10	0.76±0.09	0.074±0.039	4	0.74±0.06	0.091±0.034	2
8	1	0.73±0.11	0.130±0.142	0	0.74±0.06	0.072±0.055	0
8	5	0.80±0.08	0.075±0.059	3	0.74±0.06	0.081±0.042	0
8	10	0.79±0.11	0.069±0.063	1	0.75±0.06	0.073±0.041	1
16	1	0.68±0.17	0.112±0.108	0	0.74±0.05	0.081±0.027	0
16	5	0.77±0.11	0.115±0.061	0	0.75±0.07	0.097±0.043	0
16	10	0.76±0.11	0.102±0.047	0	0.74±0.07	0.079±0.052	0
∞	–	0.76±0.12	0.065±0.057	3	0.74±0.07	0.085±0.050	0